

Computer Architecture and Organization

Computer Architecture: Acts as the blueprint of a system, defining *what* a computer does and the attributes visible to a programmer (such as instruction sets, data types, registers, and addressing modes).

Computer Organization: Acts as the practical implementation, defining *how* the operational units and hardware components are interconnected and structured to execute that architecture.

**Architecture focuses on functional behavior and high-level design
organization focuses on structural relationships and low-level hardware details.**

Von Neumann Architecture

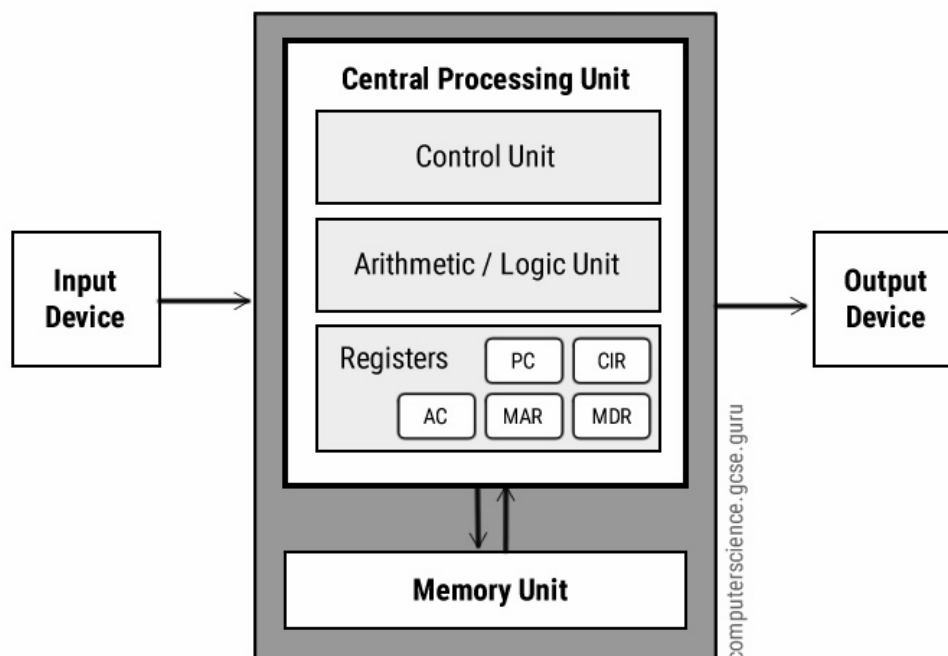
1.1 What a Computer Architect Designs

Computer architecture is the study of how the functional building blocks of a computer are organized and how they cooperate to execute programs. The designer's goal is to find a hardware/software interface – an instruction set – that makes it easy to build the hardware and the compiler while maximizing performance and minimizing cost.

1.2 The Classical Von Neumann Architecture

Von Neumann architecture was first published by John von Neumann in 1945.

Almost every general-purpose computer in use today is built on the stored-program (Von Neumann) Architecture, in which both instruction data and program data are stored in the same memory. The model is built from five essential/basic components: Control Unit, [Arithmetic and Logic Unit](#) (ALU), Memory Unit, [Registers](#) and Inputs/Outputs.



Central Processing Unit (CPU)

The [Central Processing Unit](#) (CPU) is the electronic circuit responsible for executing the instructions of a computer program.

It is sometimes referred to as the microprocessor or processor.

The CPU contains the ALU, CU and a variety of registers.

✓ **Registers**

Registers are high speed storage areas in the CPU. All data must be stored in a register before it can be processed.

MAR	Memory Address Register	Holds the memory location of data that needs to be accessed
MDR	Memory Data Register	Holds data that is being transferred to or from memory
AC	Accumulator	Where intermediate arithmetic and logic results are stored
PC	Program Counter	Contains the address of the next instruction to be executed
CIR	Current Instruction Register	Contains the current instruction during processing

✓ **Arithmetic and Logic Unit (ALU)**

The ALU allows arithmetic (add, subtract etc) and logic (AND, OR, NOT etc) operations to be carried out.

✓ **Control Unit (CU)**

The control unit controls the operation of the computer's ALU, memory and input/[output devices](#), telling them how to respond to the program instructions it has just read and interpreted from the memory unit.

The control unit also provides the timing and control signals required by other computer components.

Buses

[Buses](#) are the means by which data is transmitted from one part of a computer to another, connecting all major internal components to the CPU and memory.

A standard CPU **system bus** is comprised of a [control bus](#), [data bus](#) and [address bus](#).

Address Bus	Carries the addresses of data (but not the data) between the processor and memory
Data Bus	Carries data between the processor, the memory unit and the input/output devices
Control Bus	Carries control signals/commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer

✓ **Memory Unit**

The memory unit consists of [RAM](#), sometimes referred to as primary or main memory. Unlike a hard drive (secondary memory), this memory is fast and also directly accessible by the CPU.

RAM is split into partitions. Each partition consists of an address and its contents (both in [binary](#) form).

The address will uniquely identify every location in the memory.

Loading data from permanent memory (hard drive), into the faster and directly accessible temporary memory (RAM), allows the CPU to operate much quicker.

✓ **Inputs/Outputs**

Input unit – Brings data and programs into the machine from the outside world (keyboard, sensors, disk, network).

Output unit – Sends results from the machine back to the outside world (display, disk, network).

Source: <https://www.computerscience.gcse.guru/theory/von-neumann-architecture>

Difference between Von Neumann and (Non-Von Neumann) Harvard Architecture

<https://www.geeksforgeeks.org/computer-organization-architecture/difference-between-von-neumann-and-harvard-architecture/>

Numbers in the Computer

First, let's quickly review how a computer represents numbers. Humans are taught to think in base 10, but numbers may be represented in any base. For example, 123 base 10 = 1111011 base 2.

Numbers are kept in computer hardware as a series of high and low electronic signals, and so they are considered base 2 numbers. (Just as base 10 numbers are called *decimal* numbers, base 2 numbers are called *binary* numbers.)

Generalizing the point, in any number base, the value of i th digit d is

$$d \times \text{Base}^i$$

where i starts at 0 and increases from right to left. This leads to an obvious way to number the bits in the word: simply use the power of the base for that bit. We subscript decimal numbers with *ten* and binary numbers with *two*. For example,

1011_{two}

represents

$$\begin{aligned} & (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)_{\text{ten}} \\ &= (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{\text{ten}} \\ &= 8 + 0 + 2 + 1_{\text{ten}} \\ &= 11_{\text{ten}} \end{aligned}$$

Bit (left right)	1	0	1	1
Position (i)	3	2	1	0
Weight (2ⁱ)	8	4	2	1
Contribution	8	0	2	1

$$\text{Sum} = 8 + 0 + 2 + 1 = \mathbf{11}_{10}.$$

LSB = Least Significant Bit (rightmost, bit 0). **MSB** = Most Significant Bit (leftmost, bit 31 in a 32-bit word).

32-bit MIPS Word

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	1	

↑ MSB

LSB ↑

Signed vs. Unsigned Numbers

Sign-and-Magnitude (the Old, Abandoned Scheme)

Use the MSB as a sign bit (0 = +, 1 = -), and the remaining bits as the magnitude.

Why it was abandoned:

1. Adders need extra logic — the sign must be checked separately to know whether to add or subtract.
2. There are *two* representations of zero (+0 = 0000...0, -0 = 1000...0) — this creates bugs (e.g., `x == 0` might fail).
3. It is unclear where to put the sign bit (some early machines used the leftmost bit, others the rightmost).

Two's-Complement (the Modern Universal Scheme)

Used by MIPS (Microprocessor without Interlocked Pipelined Stages) and every modern CPU.

- If MSB = 0 value is positive; magnitude = the binary value.
- If MSB = 1 value is **negative**; magnitude found by the "invert and add 1" trick.

Complete 4-bit Two's-Complement Table

Binary	Unsigned	Two's-Complement	Formula (signed)
0000	0	0	$0 + 0 + 0 + 0$
0001	1	+1	$0 + 0 + 0 + 1$
0010	2	+2	$0 + 0 + 2 + 0$
0011	3	+3	$0 + 0 + 2 + 1$
0100	4	+4	$0 + 4 + 0 + 0$
0101	5	+5	$0 + 4 + 0 + 1$
0110	6	+6	$0 + 4 + 2 + 0$
0111	7	+7	$0 + 4 + 2 + 1$
1000	8	-8	$-8 + 0 + 0 + 0$
1001	9	-7	$-8 + 0 + 0 + 1$
1010	10	-6	$-8 + 0 + 2 + 0$
1011	11	-5	$-8 + 0 + 2 + 1$

1100	12	-4	$-8 + 4 + 0 + 0$
1101	13	-3	$-8 + 4 + 0 + 1$
1110	14	-2	$-8 + 4 + 2 + 0$
1111	15	-1	$-8 + 4 + 2 + 1$

The MSB acts as a "negative weight" — bit 31 in a 32-bit number has weight -2^{31} .

Ranges

Word Size	Unsigned Range	Two's-Complement Range
4 bits	0 to 15	-8 to +7
8 bits	0 to 255	-128 to +127
16 bits	0 to 65 535	-32 768 to +32 767
32 bits	0 to 4 294 967 295	-2 147 483 648 to +2 147 483 647
n bits (general)	0 to $2^n - 1$	-2^{n-1} to $+2^{n-1} - 1$

The Negation Rule — "Invert and Add 1"

1. **Invert** every bit (0 → 1).
2. **Add 1**.

Worked Example 1-A — Negate +2 in 32-bit two's complement

```

+2 :    0000 0000 0000 0000 0000 0000 0000 0010
~   :    1111 1111 1111 1111 1111 1111 1111 1101    (invert)
+ 1 :                                           1
-----
-2 :    1111 1111 1111 1111 1111 1111 1111 1110

```

Sanity check. Negate -2 by the same rule and we should get +2:

```

-2 :    1111 1111 1111 1111 1111 1111 1111 1110
~   :    0000 0000 0000 0000 0000 0000 0000 0001
+ 1 :                                           1
-----
+2 :    0000 0000 0000 0000 0000 0000 0000 0010  ✓

```

Worked Example 1-B — Negate +100 (8-bit)

+100 = 0110 0100; invert 1001 1011; add 1 1001 1100 = **-100**.

Sanity check: 1001 1100 read as signed = $-128 + 16 + 8 + 4 = -100$ ✓.

Sign Extension

